Day 78 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 12, Day 2 reference

Watch: Encoder Architecture in Transformers, Step by Step Guide (Learn With Jay) | PostLN, PreLN and ResiDual Transformers (Machine Learning Studio)

`day78_encoder_block.py`, segment by segment, in the order `main()` uses things. The attention functions (Days 75–76), the encoding (Day 77) and the data code (Day 62) are copied verbatim and explained there. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: x = a position's vector (d = 32 numbers) · μ, σ = its mean and spread · g, b = LayerNorm's learned scale and shift · F = 128, the FFN's width · h1 = the block's middle (after attention)

Segment 0 — Setup

Settings

CODE

```
SEEDS = [0, 1, 2]
EPOCHS = 25
LRS = (0.5, 1.0, 2.0)
D, F, HEADS = 32, 128, 4
PE_SCALE = 3.0
MAX_LEN = 128
```

- `D, F, HEADS = 32, 128, 4` — three names from one tuple; F = 4 D as in the paper.
- `PE_SCALE = 3.0` — Day 77's choice on validation, used here without retuning.

Segment 1 — The new parts

layer_norm and its backward pass

MATH

$$y = (x - \mu) / \sqrt{(\sigma^2 + \varepsilon)} \cdot g + b$$

$$z = (x - \mu) / \sigma, \quad u = dy \cdot g, \quad dx = (u - mean(u) - z \cdot mean(u\, z)) / \sigma$$

CODE

```
def layer_norm(x, g, b, eps=1e-5):
    mu = x.mean(axis=-1, keepdims=True)
    sd = np.sqrt(x.var(axis=-1, keepdims=True) + eps)
    xhat = (x - mu) / sd
    return xhat * g + b, (xhat, sd)
def layer_norm_backward(dy, g, cache):
    xhat, sd = cache
    dxhat = dy * g
    dx = (dxhat - dxhat.mean(axis=-1, keepdims=True) - xhat * (dxhat * xhat).mean(axis=-1,
        keepdims=True)) / sd
    flat = lambda T: T.reshape(-1, T.shape[-1])
    return dx, flat(dy * xhat).sum(axis=0), flat(dy).sum(axis=0)
```

- `x.mean(axis=-1, keepdims=True)` — one mean per position; keepdims keeps a size-1 last axis so it broadcasts back.
- `np.sqrt(x.var(...) + eps)` — eps keeps the spread above 0 for a vector whose numbers are all equal.

- `(dxhat - mean(dxhat) - xhat * mean(dxhat * xhat)) / sd` — the gradient through the mean and the spread, in one line.

ffn and its backward pass

MATH

$FFN(h) = ReLU(h\,W_1 + c_1)\,W_2 + c_2$, *the same weights at every position*

CODE

```
def ffn(x, P):
    u = x @ P["W1"] + P["c1"]
    r = np.maximum(u, 0.0)
    return r @ P["W2"] + P["c2"], (x, u, r)
def ffn_backward(dy, P, cache):
    x, u, r = cache
    flat = lambda T: T.reshape(-1, T.shape[-1])
    du = (dy @ P["W2"].T) * (u > 0)
    g = dict(W2=flat(r).T @ flat(dy), c2=flat(dy).sum(axis=0), W1=flat(x).T @ flat(du),
        c1=flat(du).sum(axis=0))
    return du @ P["W1"].T, g
```

- `np.maximum(u, 0.0)` — ReLU: negative numbers become 0.
- `(dy @ P["W2"].T) * (u > 0)` — the ReLU passes gradient only where its input was positive; (u > 0) is a boolean array used as 0/1.
- `x @ P["W1"]` — x has shape (m, T, d): matmul applies the same W1 to every position.

Segment 2 — The block

EncoderBlock.__init__ and the norm helpers

CODE

```
class EncoderBlock:
    def __init__(self, rng, pre_ln=False, residual=True, norm=True, use_ffn=True):
        d, f, heads = D, F, HEADS
        self.h, self.pre, self.res, self.norm, self.use_ffn = heads, pre_ln, residual, norm,
            use_ffn
        s = lambda a, b: rng.randn(a, b) / np.sqrt(a)
        self.P = dict(Wq=s(d, d), Wk=s(d, d), Wv=s(d, d), Wo=s(d, d),
                      g1=np.ones(d), b1=np.zeros(d),
                      W1=s(d, f), c1=np.zeros(f), W2=s(f, d), c2=np.zeros(d),
                      g2=np.ones(d), b2=np.zeros(d))
    def _norm(self, x, which):
        if not self.norm:
            return x, None
        return layer_norm(x, self.P["g" + which], self.P["b" + which])
    def _norm_back(self, dy, which, cache, g):
        if not self.norm:
            return dy
        dx, g["g" + which], g["b" + which] = layer_norm_backward(dy, self.P["g" + which], cache)
        return dx
```

- `d, f, heads = D, F, HEADS` — read the sizes when the block is made, so the gradient check's tiny sizes take effect.
- `s = lambda a, b: ...` — a small local function that makes one weight matrix.
- `g1=np.ones(d), b1=np.zeros(d)` — LayerNorm starts as 'no change': scale 1, shift 0.
- `return x, None` — the 'no norm' ablation returns its input and an empty cache.

EncoderBlock.forward

MATH

post-LN: h1 = LN(x + Attn(x)), y = LN(h1 + FFN(h1)); pre-LN: h1 = x + Attn(LN(x)), y = h1 + FFN(LN(h1))

CODE

```
def forward(self, x, pe, kmask):
    P, h, c = self.P, self.h, {}
    a_in, c["n1"] = self._norm(x, "1") if self.pre else (x, None)
    Xqk = a_in + pe
    Q, K, V = split_heads(Xqk @ P["Wq"], h), split_heads(Xqk @ P["Wk"], h), split_heads(a_in @
        P["Wv"], h)
    Zh, A = scaled_dot_product_attention(Q, K, V, kmask)
    Z = merge_heads(Zh, h)
    s1 = Z @ P["Wo"] + (x if self.res else 0.0)
    h1, n1 = (s1, None) if self.pre else self._norm(s1, "1")
    c["n1"] = c["n1"] if self.pre else n1
    c.update(a_in=a_in, Xqk=Xqk, Q=Q, K=K, V=V, A=A, Z=Z)
    if not self.use_ffn:
        self.c = c
        return h1
    f_in, c["n2"] = self._norm(h1, "2") if self.pre else (h1, None)
    fo, c["ffn"] = ffn(f_in, P)
    s2 = fo + (h1 if self.res else 0.0)
    y, n2 = (s2, None) if self.pre else self._norm(s2, "2")
    c["n2"] = c["n2"] if self.pre else n2
    self.c = c
    return y
```

- `... if self.pre else (x, None)` — a conditional expression that returns a pair either way.
- `Z @ P["Wo"] + (x if self.res else 0.0)` — the residual: the block's input is added back; 0.0 for the ablation.
- `c.update(a_in=a_in, ...)` — adds several cache entries at once.

EncoderBlock.backward

MATH

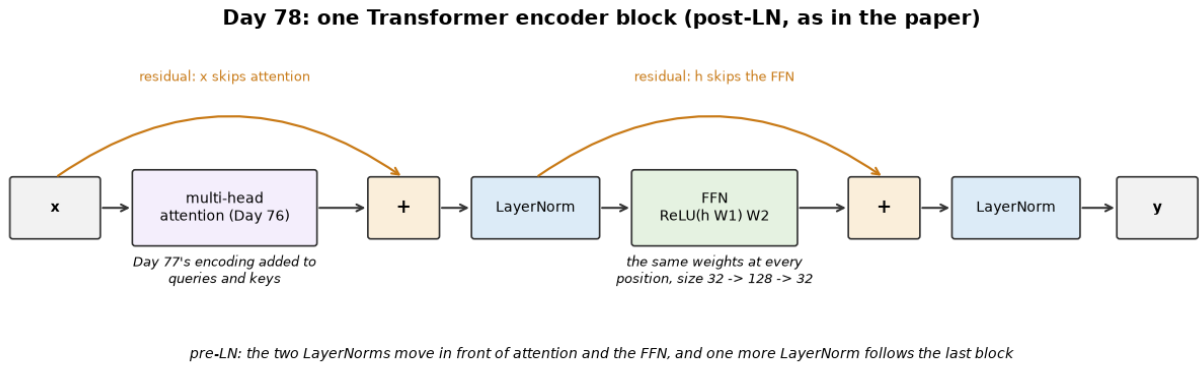
$s = x + f(x) \implies ds/dx = I + df/dx$: the identity path carries the gradient whatever f does

CODE

```
def backward(self, dy):
    P, h, c = self.P, self.h, self.c
    g = {k: np.zeros_like(v) for k, v in P.items()}
    flat = lambda T: T.reshape(-1, T.shape[-1])
    if self.use_ffn:
        ds2 = dy if self.pre else self._norm_back(dy, "2", c["n2"], g)
        df_in, gf = ffn_backward(ds2, P, c["ffn"])
        g.update(gf)
        if self.pre:
            df_in = self._norm_back(df_in, "2", c["n2"], g)
        dh1 = df_in + (ds2 if self.res else 0.0)
    else:
        dh1 = dy
    ds1 = dh1 if self.pre else self._norm_back(dh1, "1", c["n1"], g)
    g["Wo"] = flat(c["Z"]).T @ flat(ds1)
    dQ, dK, dV = scaled_dot_product_attention_backward(split_heads(ds1 @ P["Wo"].T, h), c["Q"],
        c["K"], c["V"], c["A"])
    dQ, dK, dV = merge_heads(dQ, h), merge_heads(dK, h), merge_heads(dV, h)
    g["Wq"], g["Wk"], g["Wv"] = flat(c["Xqk"]).T @ flat(dQ), flat(c["Xqk"]).T @ flat(dK),
        flat(c["a_in"]).T @ flat(dV)
    da_in = dQ @ P["Wq"].T + dK @ P["Wk"].T + dV @ P["Wv"].T
    if self.pre:
        da_in = self._norm_back(da_in, "1", c["n1"], g)
    return da_in + (ds1 if self.res else 0.0), g
```

- `{k: np.zeros_like(v) for k, v in P.items()}` — every weight starts with a zero gradient, so an ablated part reports zeros.
- `dh1 = df_in + (ds2 if self.res else 0.0)` — the residual: the gradient reaches h1 through the FFN AND directly.
- `return da_in + (ds1 if ...)` — the same at the bottom: through attention AND directly.

SYSTEM DIAGRAM



The post-LN encoder block.

Segment 3 — The tagger

EncoderTagger: params, forward, log-softmax

MATH

$$\log p_k = z_k - \max z - \log \Sigma_j \exp(z_j - \max z)$$

CODE

```
def params(self):
    out = dict(self.P)
    for i, b in enumerate(self.blocks):
        out.update({f"{i}.{k}": v for k, v in b.P.items()})
    return out
def forward(self, ids):
    m, T = ids.shape
    self.ids = ids
    mask = ids != PAD
    kmask = np.repeat(mask[:, None, None, :], HEADS, axis=1).reshape(m * HEADS, 1, T)
    x = self.P["E"][ids]
    for i, b in enumerate(self.blocks):
        pe = self.PE[:T][None] if (i == 0 or self.pe_every) else 0.0
        x = b.forward(x, pe, kmask)
    if self.pre:
        x, self.nf = layer_norm(x, self.P["gf"], self.P["bf"])
    self.y = x
    logits = x.reshape(m * T, -1) @ self.P["Wout"] + self.P["bout"]
    self.logp = logits - logits.max(axis=1, keepdims=True)
    self.logp -= np.log(np.exp(self.logp).sum(axis=1, keepdims=True))
    return self.logp
```

- `{f"{i}.{k}": v for k, v in b.P.items()}` — a dictionary comprehension with f-string keys such as '0.Wq'.
- `pe = ... if (i == 0 or self.pe_every) else 0.0` — positions in every block, or only in the first (Section 7).
- `logits - logits.max(...)` — then minus the log of the summed exponentials: log-softmax without ever taking log(p).

gradients and train_step

CODE

```
def gradients(self, tags, mask):
    m, T = self.ids.shape
    dlog = (np.exp(self.logp) - np.eye(self.n_tags)[tags.ravel()]) * mask.ravel()[:, None] /
        mask.sum()
    G = dict(Wout=self.y.reshape(m * T, -1).T @ dlog, bout=dlog.sum(axis=0))
    dx = (dlog @ self.P["Wout"].T).reshape(m, T, -1)
    if self.pre:
        dx, G["gf"], G["bf"] = layer_norm_backward(dx, self.P["gf"], self.nf)
    for i in reversed(range(len(self.blocks))):
        dx, g = self.blocks[i].backward(dx)
        G.update({f"{i}.{k}": v for k, v in g.items()})
    G["E"] = np.zeros_like(self.P["E"]); np.add.at(G["E"], self.ids, dx)
    return G
def train_step(self, ids, tags, mask):
    self.forward(ids)
    G, P = self.gradients(tags, mask), self.params()
    norm = np.sqrt(sum(np.sum(v ** 2) for v in G.values()))
    for k, v in G.items():
        P[k] -= self.lr * min(1.0, self.clip / norm) * v
```

- `np.exp(self.logp) - np.eye(self.n_tags)[...]` — softmax minus the one-hot true tag: the cross-entropy gradient.

- `for i in reversed(range(len(self.blocks)))` — the last block first, as the chain rule requires.
- `P[k] -= ...` — in place: P holds the very arrays inside the blocks, so the blocks are updated too.

Segment 4 — Experiments

fit: restoring the best snapshot

CODE

```
for k, v in best_P.items():
    net.params()[k][...] = v
acc = lambda split: token_accuracy(net.predict(split[0]), split[1], split[2])
return dict(net=net, val=best_val, epoch=best_epoch, test=acc(data[2]), train=acc(data[0]))
```

- `net.params()[k][...] = v` — [...] = writes into the existing array, so every reference to it sees the snapshot.
- `acc = lambda split: ...` — a short function used twice: test and training accuracy.

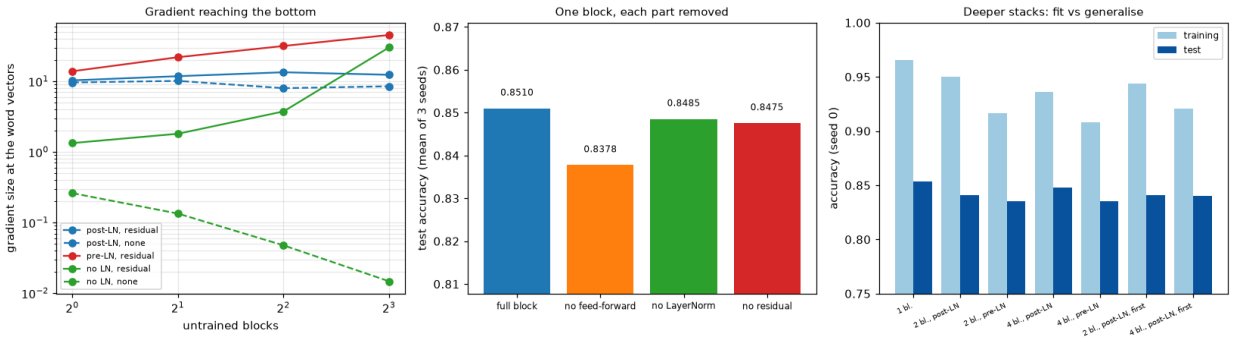
demo_residual: the five versions

CODE

```
kinds = [("post-LN, residual", dict()), ("post-LN, none", dict(residual=False)), ("pre-LN,
    residual", dict(pre_ln=True)),
         ("no LN, residual", dict(norm=False)), ("no LN, none", dict(norm=False, residual=False))]
print(f"\n{'blocks':>7}" + "".join(f"{k:>19}" for k, _ in kinds))
res = {k: [] for k, _ in kinds}
for n in (1, 2, 4, 8):
    for label, kw in kinds:
        net = EncoderTagger(len(itos), 17, n_blocks=n, seed=0, **kw)
        net.P["E"] = np.random.RandomState(2).randn(*net.P["E"].shape) * 0.1
        x = net.P["E"][b]
        kmask = np.repeat(real[:, None, None, :], HEADS, axis=1).reshape(len(b) * HEADS, 1, T)
        for blk in net.blocks:
            x = blk.forward(x, net.PE[:T][None], kmask)
        dx = top
        for blk in reversed(net.blocks):
            dx, _ = blk.backward(dx)
        res[label].append(np.sqrt((dx ** 2).sum(axis=-1))[real].mean())
```

- `dict(norm=False, residual=False)` — keyword arguments packed in a dictionary and passed on with **kw.
- `np.sqrt((dx ** 2).sum(axis=-1))[real]` — the length of each word's gradient vector, real words only.

REAL GRAPH



Real graph from the script: gradient by depth, each part removed, deeper stacks.

Quick Syntax Glossary — new or reinforced constructs

- `x.mean(axis=-1, keepdims=True)` — statistics over the last axis that still broadcast against x.
- `np.maximum(u, 0.0)` — ReLU, elementwise.
- `(u > 0)` — a boolean array, used as 0/1 in arithmetic.
- `a if cond else b` — a conditional expression; used here to switch parts off.
- `global D, F` — rebind module-level names inside a function (the gradient check's tiny sizes).
- `arr[...] = v` — write into an existing array in place.
- `reversed(range(n))` — n-1, ..., 0.

End of Day 78 syntax reference. For the experiments and what they showed, see the Day 78 complete guide.